

SCANDEF file

Wednesday, September 16, 2026 9:40 AM

What Is a .scandef File ?

Scan Design Exchange Format for DFT and Physical Design

"A bridge between DFT and Physical Design to enable testable, high-quality silicon."

- Definition**
 - SCANDEF (Scan Design Exchange Format) is a file format that describes how scan flip-flops are connected into scan chains.
 - It is an extension of the LEF/DEF family (uses similar syntax style).
 - It contains **only test structure information (scan chains)** and is used in the DFT and physical design flow.
- Why Does It Exist ?**
 - After DFT (scan insertion), the design contains many scan flip-flops (SFFs).
 - Physical design tools need to know the scan chain connectivity to:
 - place and reorder scan chains
 - reduce routing congestion
 - improve test performance (shorter wires)
 - SCANDEF provides this information in a standard format so that different tools can exchange it.
- Need for Scan Chains (Post-Manufacture)**
 - After manufacturing, we cannot easily access internal signals of the chip.
 - Scan chains allow us to shift test data through all scan flip-flops and verify that each flip-flop and the surrounding logic is working correctly.
 - Without scan chains, it is very difficult (or impossible) to test millions of flip-flops inside the chip.
 - SCANDEF is required to generate test patterns (ATPG) and to implement the scan chains physically.
- Scan Flip-Flop Concept**

Functional Mode (scan_en = 0)
 Flip-flop behaves normally.
 D input is active.

Shift Mode (scan_en = 1)
 Flip-flop ignores D input.
 SI input takes over.
- Scan Chain Idea**

In test mode, all scan flip-flops are connected in series to form a shift register.

 - Shift in test data through SI.
 - Capture responses from SO.
 - All flip-flops are tested serially (or in parallel using multiple chains).
- Role of .scandef in the Design Flow**
- What Does SCANDEF Describe ?**
 - List of scan chains in the design
 - Ordered list of scan flip-flops in each chain
 - Start and stop points (SI / SO)
 - Scan chain attributes (e.g., floating, ordered, partition)
 - Scan groups (optional, tool dependent)
 - Common scan pins (e.g., scan_enable)
 - Information needed for physical placement and reordering of scan chains
- What Does SCANDEF NOT Describe ?**
 - Detailed routing (wire shapes, vias)
 - Cell placement coordinates (that's in .def)
 - Functional logic connectivity (beyond scan chain info)
 - Timing, delays, or test patterns (that's ATPG output)
 - Power, clock, or regular signal nets (except scan-related pins if specified)
 - Physical layout details (metal layers, shapes, etc.)

Key Takeaway: SCANDEF is a physical-design exchange representation of scan-chain structure, particularly useful for scan-chain placement and reordering.

- Enables testability
- Improves physical implementation
- Helps deliver high-quality, defect-free silicon

How a Scan Flip-Flop Actually Works ?

Same cell. Two modes. One extra input. Big benefit for testability.

- Normal Flip-Flop (Without Scan)**
 - Single data input (D).
 - On every clock edge, D is captured and appears at Q.
 - Used in normal functional operation.
 - Not directly controllable/observable for manufacturing test.
- Scan Flip-Flop Structure**

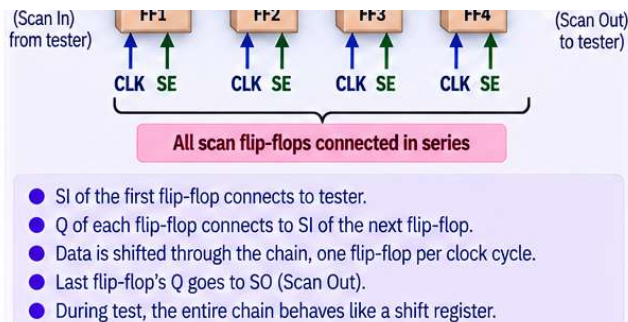
Only difference is the MUX in front!

 - A 2:1 MUX selects between D and SI based on scan enable (SE).
 - When SE = 0 → normal functional mode (D is selected).
 - When SE = 1 → scan mode (SI is selected).
 - Q output goes to functional logic (in functional mode) or to next scan flip-flop (in scan mode).
- Modes of Operation**

Functional Mode (SE = 0)
 D is selected.

Shift Mode (SE = 1)
 SI is selected.

 - Flip-flop behaves normally.
 - D input is captured on clock edge and goes to Q.
 - Chip works in its real functional mode (application mode).
 - Flip-flop is used during manufacturing test.
- Scan Chain Connection (Shift Register)**
- Test Operations**
 - Shift Operation (Load Test Data)
 - Set SE = 1 (scan mode). Used to load test.
- Why This Matters?**
 - Helps test millions of flip-flops
 - Enables detection of defects in manufactured silicon



- Apply clock pulses.
 - Shift test data through SI → SO.
 - Each clock moves one bit to the next flip-flop.
- pattern into all scan flip-flops.
- 2. Capture Operation (Capture Circuit Response)**
- Set SE = 0 (functional mode).
 - Apply functional inputs to the chip.
 - Apply one/few clock cycles to capture responses in each flip-flop.
 - Set SE = 1 again and shift out the captured data through SO.
- Used to capture the actual output of the logic into scan flip-flops.

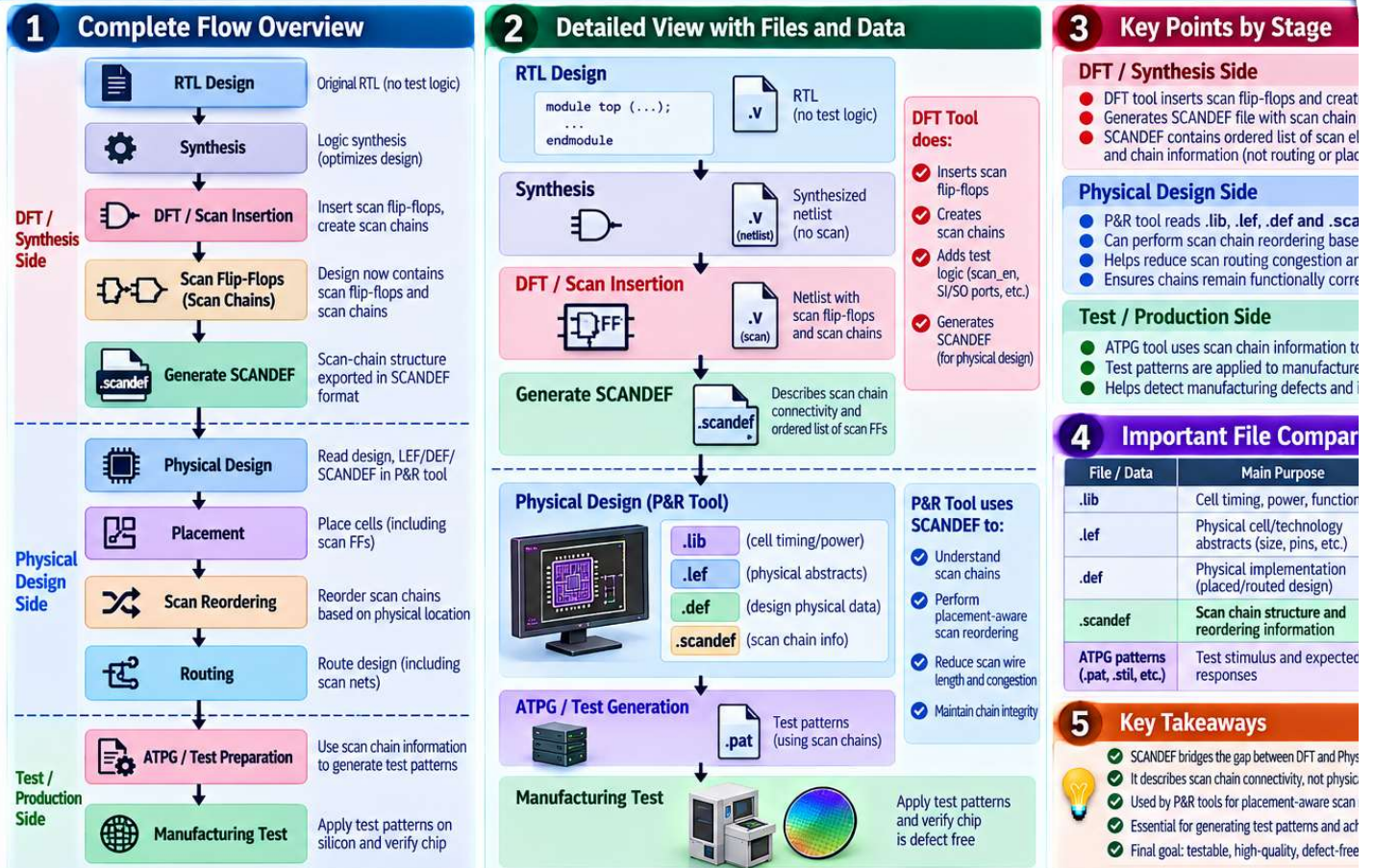
- sequential logic.
- ✓ Allows controllability and observability.
 - ✓ SCANDEF lists the ordered scan chain, so tools can generate test patterns and place/reorder chains during routing.
 - ✓ The order of flip-flops in the scan chain is important, that's why SCANDEF must be used.

Key Takeaway:

A scan flip-flop is a normal MUX. With SE = 0 it works in functional mode. With SE = 1 it becomes part of the scan chain.

Where SCANDEF Comes From in the DFT → Physical Design Flow

From RTL to Silicon – SCANDEF bridges DFT and Physical Design for a testable chip.



SCANDEF does not replace DEF. Both are needed — one for physical implementation (.def) and one for scan chain structure (.scandef).

Right Information at the Right Stage →

SCANDEF HEADER & BASIC SYNTAX

Reading the file header before entering SCANCHAINS

Correct syntax
Connects DFT to Physical Design
Enables a Testable Silicon!



Example: SCANDEF Header (Partial)

```
1 VERSION 5.5 ;
2 NAMESCASESENSITIVE ON ;
3 DIVIDERCHAR "/" ;
4 BUSBITCHARS "[" ;
5
6 DESIGN my_design ;
7
8 SCANCHAINS 2 ;
9 ...
10 END SCANCHAINS
```

Typical SCANDEF header

1 VERSION

VERSION 5.5 ;

- Specifies the SCANDEF format version.
- Tool uses this to check compatibility while reading the file.
- Different tools may support different versions.

2 NAMESCASESENSITIVE

NAMESCASESENSITIVE ON ;

- When ON, instance names, net names, etc. are case-sensitive.
- top/u_ff and top/U_FF are different instances.
- Recommended to keep it ON to avoid name mismatches.

3 DIVIDERCHAR

DIVIDERCHAR "/" ;

- Defines the hierarchy character.
- Here "/" is used.
- Example: top/instance
- Must match the netlist and DEF.

4 BUSBITCHARS

BUSBITCHARS "[" ;

- Defines the characters used for bus bit notation.
- Here "[" is used.

5 DESIGN

DESIGN my_design ;

- Specifies the top design name.
- Must match EXACTLY with:
 - Top module name in RTL
 - Top module name in case-insensitive netlist

6 SCANCHAINS

SCANCHAINS 2 ;

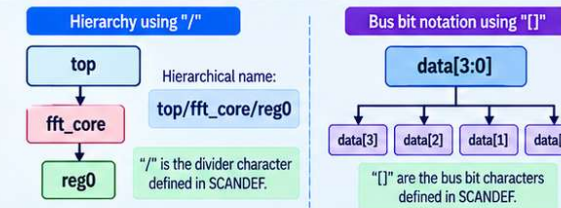
- Declares the number of scan chains in the design (here 2).
- This starts the list of scan chains where each chain is listed.

- Example: data[3], addr[7:0]
- Must be consistent with the netlist/DEF naming.

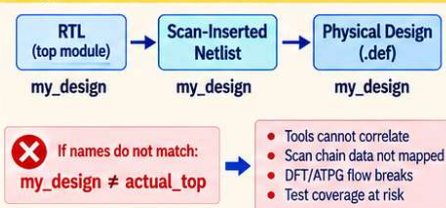
- Top module name in scan-inserted netlist
- DESIGN name in .def file
- If mismatched, tools cannot correlate scan chains to the design.

7 END SCAN
END SCANC
Closes the scan-

Hierarchy and Bus Notation Example



Why DESIGN Name Matters ?



VERSION Note (Important)

- LEF/DEF standards have multiple released versions (including 5.8 and 6.x).
- ScanDEF examples can be tool/version dependent.
- Treat VERSION 5.5 here as an illustrative example, not a universal requirement.
- Refer to tool documentation for the support.

What Comes Next ?

Inside SCANCHAINS section
START / STOP / scan elements (list)

Key Takeaway: The SCANDEF header defines the format, naming rules and design context. It must be consistent with your RTL, netlist and DEF for a successful DFT and physical design flow.

Right Information → Test



SCANCHAINS: The Heart of SCANDEF

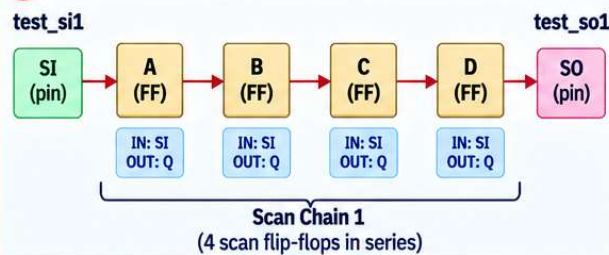
Describes the ordered list of scan flip-flops and how they are connected in each chain.

1 Example: SCANCHAINS Section

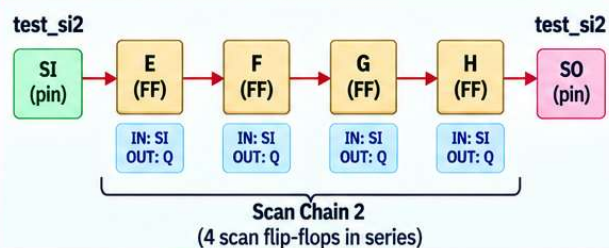
```

1 SCANCHAINS 2 ;
2
3 - chain1
4 + START PIN test_si1
5 + FLOATING
6 A ( IN SI ) ( OUT Q )
7 B ( IN SI ) ( OUT Q )
8 C ( IN SI ) ( OUT Q )
9 D ( IN SI ) ( OUT Q )
10 + STOP PIN test_so1 ;
11
12 - chain2
13 + START PIN test_si2
14 + FLOATING
15 E ( IN SI ) ( OUT Q )
16 F ( IN SI ) ( OUT Q )
17 G ( IN SI ) ( OUT Q )
18 H ( IN SI ) ( OUT Q )
19 + STOP PIN test_so2 ;
20
21 END SCANCHAINS
  
```

2 Visual Representation of Chain1



Visual Representation of Chain2



3 Key Syntax Elements

- Chain name (- chain1)**
 - Unique name to identify
 - Referenced in other sections
- START**
 - Specifies the start point
 - Must match a scan input
- PIN**
 - Indicates that the start/stop is a pin
 - Can also be an internal node
- FLOATING**
 - Allows the tools to reorder elements in this chain (placement)
 - Alternative: ORDERED
- Instance name (A, B, C)**
 - Name of the scan flip-flop
 - Must match exactly with netlist (.v) and DEF (CO)
- (IN SI)**
 - Specifies the scan input
 - Typically "SI" as defined
- (OUT Q)**
 - Specifies the scan output
 - Typically "Q" (can be different)
- STOP**
 - Specifies the end point
 - Must match a scan output
- Ordering of scan elements**
 - The order (A → B → C → D)
 - ATPG uses this order to shift

4 Why Instance Names Must Match the Actual Design ?

- SCANDEF instance names must be the same as in:
 - RTL netlist (.v)
 - Scan-inserted netlist
 - DEF file (COMPONENTS section)
- If names do not match, tools cannot locate the flip-flops.
- This will break scan chain mapping, reordering, and test pattern generation.

! Name mismatch
= chain cannot be built
= ATPG will fail
= no test coverage

5 Key Takeaways

- ✓ SCANCHAINS defines the ordered list of scan elements and their connectivity.
- ✓ Each chain has a START and STOP (usually top pins).
- ✓ FLOATING allows physical reordering for better placement and routing.
- ✓ Instance names and pin names must match the actual design.
- ✓ Correct ordering is essential for successful ATPG and testing.

Remember:

SCANCHAINS is the heart of SCANDEF – it tells the tools exactly which flip-flops are in each scan chain and in what order.

Correct scan chain description → Better place
→ Faster test → Higher yield !

FLOATING, ORDERED, PARTITION, MAXBITS & Scan Reordering

Key SCANDEF constructs that enable physically aware scan chain reordering in Physical Design

1 FLOATING

Elements can be reordered by the tool

- FLOATING allows the physical design tool to change the order of scan elements in this chain to improve placement and routing.
- Tool can reorder to reduce wire length, congestion and improve test performance.



2 ORDERED

Preserve the given order

- ORDERED keeps the relative order of scan elements fixed.
- Tool cannot arbitrarily change the order of elements in this section.
- Used when reordering is not allowed (e.g., due to test constraints or logic dependencies).



3 PARTITION

Group chains that can be reordered together

- A partition is a group of scan chains that may exchange flip-flops during reordering.
- Chains in the same partition typically share the same clock domain or test constraints.
- Helps the tool to optimize globally across multiple chains.

Example: Partition CLK_A



4 MAXBITS

Limit chain size

- MAXBITS specifies the maximum number of scan elements in a chain segment.
- Helps control chain time, routing, and
- If the chain exceeds MAXBITS, the tool will split the chain or re

Example

MAXBI

(possible with FLOATING)

```
graph LR
  A --> C --> B --> D
```

✓ **Relative order can change.**
Tool chooses the best order based on physical location.

After Physical Design

```
graph LR
  A --> B --> C --> D
```

✗ **Order must be preserved.**
Tool cannot change the relative order.

reordering

```
graph LR
  Chain3 --> Chain4 --> Different_partition["Different partition (e.g., CLK_B)"]
```

✓ **Chains in the same partition can exchange flip-flops during reordering.**

- Limits each chain scan flip-flops.
- Tool will split long if needed.
- Useful to balance improve test time

5 Why Reorder Scan Chains ?

Physical location vs. logical order

Original logical order (from DFT)

```
graph LR
  SI --> FF1 --> FF2 --> FF3 --> FF4 --> SO
```

Possible physical locations on chip

If we keep original order:
Long scan wires, more congestion and crossings.

After reordering (e.g., FF1 → FF3 → FF2 → FF4):
Shorter wires, easier routing, less congestion, better PPA.

6 Benefits of Scan Reordering

- ✓ Reduces scan-wire length
- ✓ Lower routing congestion
- ✓ Easier routing and fewer layer crossings
- ✓ Improves timing and signal integrity
- ✓ Helps achieve better placement
- ✓ Can reduce test time (with balanced chains)
- ✓ Improves overall design PPA (performance, power, area)

"Same test logic, smarter physical implementation with SCANDEF reordering."

7 Key Points to Remember

- ✓ **FLOATING** → Tool can reorder
- ✗ **ORDERED** → Tool must preserve
- ✓ **PARTITION** → Chains in same partition (usually same clock)
- ✓ **MAXBITS** → Limits chain length
- ✓ Reordering is based on physical wire length and congestion.
- ✓ **Goal:** better routability, shorter test time, higher quality silicon.

Key Takeaway: **FLOATING, ORDERED, PARTITION** and **MAXBITS** give the physical design tool the flexibility and constraints it needs to optimally reorder scan chains for a clean, testable and high-quality implementation.

COMPLETE REALISTIC SCANDEF FILE

A teaching example — read the file from header → scan chains → physical interpretation

From Text to Testable Silicon!

```
mimo_top.scandef
1  VERSION 5.5 ;
2  NAMESCASESENSITIVE ON ;
3  DIVIDERCHAR "/" ;
4  BUSBITCHARS "[]" ;
5
6  DESIGN mimo_top ;
7
8  SCANCHAINS 2 ;
9
10 - chain1
11 + START PIN scan_in1
12 + FLOATING
13   fft_core/reg0 ( IN SI ) ( OUT Q )
14   fft_core/reg1 ( IN SI ) ( OUT Q )
15   fft_core/reg2 ( IN SI ) ( OUT Q )
16   fft_core/reg3 ( IN SI ) ( OUT Q )
17 + PARTITION clk_rf
18 + STOP PIN scan_out1 ;
19
20 - chain2
21 + START PIN scan_in2
22 + FLOATING
23   channel_est/reg0 ( IN SI ) ( OUT Q )
24   channel_est/reg1 ( IN SI ) ( OUT Q )
25   equalizer/reg0 ( IN SI ) ( OUT Q )
26   equalizer/reg1 ( IN SI ) ( OUT Q )
27 + PARTITION clk_dsp
28 + STOP PIN scan_out2 ;
29
30 END SCANCHAINS
```

This is a realistic and simplified SCANDEF example using commonly supported 5.5 syntax.

1 HEADER INFORMATION

- **VERSION 5.5 ;** ScanDEF format version. Tool checks compatibility.
- **NAMESCASESENSITIVE ON ;** Keeps instance/net names case sensitive.
- **DIVIDERCHAR "/" ;** Hierarchy separator used in instance names (e.g., fft_core/reg0).
- **BUSBITCHARS "[]" ;** Bus bit notation (e.g., data[3]).
- **DESIGN mimo_top ;** Top design name. Must match EXACTLY with the RTL, scan-inserted netlist and DEF.

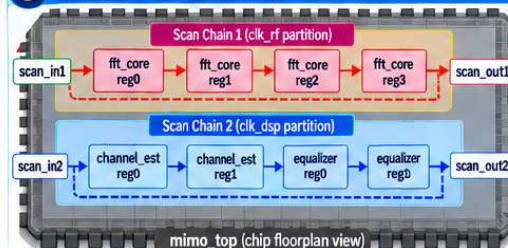
3 CHAIN 1 DETAILS

- **- chain1** Chain name (unique identifier).
- **+ START PIN scan_in1**
- **+ FLOATING** Start point (top-level pin).
- Scan elements in this chain (can be reordered).
 - fft_core/reg0 (IN SI) (OUT Q)
 - fft_core/reg1 (IN SI) (OUT Q)
 - fft_core/reg2 (IN SI) (OUT Q)
 - fft_core/reg3 (IN SI) (OUT Q)
- **+ PARTITION clk_rf**
- Chain belongs to partition 'clk_rf'.
- **+ STOP PIN scan_out1 ;** End point (top-level pin).

4 CHAIN 2 DETAILS

- **- chain2** Chain name (unique identifier).
- **+ START PIN scan_in2**
- **+ FLOATING** Start point (top-level pin).
- Scan elements in this chain (can be reordered).
 - channel_est/reg0 (IN SI) (OUT Q)
 - channel_est/reg1 (IN SI) (OUT Q)
 - equalizer/reg0 (IN SI) (OUT Q)
 - equalizer/reg1 (IN SI) (OUT Q)
- **+ PARTITION clk_dsp**
- Chain belongs to partition 'clk_dsp'.
- **+ STOP PIN scan_out2 ;** End point (top-level pin).

8 PHYSICAL VIEW OF THE EXAMPLE DESIGN



DO NOT CONFUSE

- SCANDEF** = Scan-chain structure / reordering information
- DEF** = Physical implementation (cell locations, routing, etc.)
- ATPG** = Test patterns (stimulus and expected responses)

5 WHAT FL

- Elements listed u reordered by the (placement-aware)
- Helps reduce wire and improves tes
- Tool maintains co within allowed co

6 WHAT PA

- A partition is a gr exchange flip-flop
- Chains with the s can participate in
- Helps the tool op multiple chains.

7 INSTANCE

- Instance names (exist in the actu
- Pin names (IN SI scan cell definiti
- If names do not n scan chains and

DO NOT C

- SCANDEF** = Scan-chain
- DEF** = Physical in
- ATPG** = Test patte



Key Takeaway:

A realistic SCANDEF connects DFT-created scan-chain knowledge to physical implementation; the physical tool uses the chain information for placement-aware reordering while the DEF records the resulting physical design.

Teaching example based on exact tool/version output



SCANDEF File – Complete Example (Part 1/3)

Header Section – Format, Naming Rules and Design Declaration

Defin of the Mus with

mimo_top.scandef (Part 1/3 – Header Section)

Lines 1 – 25

```
1 #=====
2 # SCANDEF Example (Part 1/3)
```

1 File Header Explanation

Statement	Example	Meaning / Purpose
VERSION	VERSION 5.5 ;	• Specifies the SCANDEF • Tool checks compatibil

```

3 # Design      : mimo_top
4 # Description  : Scan chain information for DFT and P&R
5 #=====
6
7 VERSION 5.5 ;
8
9 # Case sensitivity for instance and net names
10 NAMESCASESENSITIVE ON ;
11
12 # Hierarchy separator used in instance names
13 DIVIDERCHAR "/" ;
14
15 # Characters used for bus notation
16 BUSBITCHARS "[" ;
17
18 # Top design name (must match RTL, netlist and DEF)
19 DESIGN mimo_top ;
20
21 #-----
22 # Start of scan chain description
23 #-----
24 SCANCHAINS 2 ; # Total number of scan chains in this design
25

```



This is Part 1 of 3. It contains the file header and the start of the SCANCHAINS section. Part 2 will show the actual chain definitions and Part 3 will show the remaining chains and EOF.

NAMESCASESENSITIVE	NAMESCASESENSITIVE ON ;	- When ON, instance names are case sensitive. - Recommended to keep.
DIVIDERCHAR	DIVIDERCHAR "/" ;	- Defines the hierarchy separator. - Here "/" is used (e.g., fft_core/butterfly_reg_0).
BUSBITCHARS	BUSBITCHARS "[" ;	- Defines the characters used for bus notation. - Here "[" is used (e.g., fft_core/butterfly_reg_0[0]).
DESIGN	DESIGN mimo_top ;	- Top design name. - Must match EXACTLY with: - Top module name - Top module name - DESIGN name in .def
SCANCHAINS	SCANCHAINS 2 ;	- Declares the number of scan chains. - Each chain will be listed.

2 Important Notes

- Version** Different EDA tools may support different versions (e.g., 5.5). Always use a version supported by your tool.
- Naming Consistency** The DESIGN name and all instance names must match the netlist. Mismatch can cause the tool to fail in chain correlation.
- Hierarchy and Bus Notation** Make sure DIVIDERCHAR and BUSBITCHARS are consistent with your design.
- Number of Chains** The number specified in SCANCHAINS must match the actual number of chains in the design.

3 What's Next?

- In Part 2, we will define Chain 1 in detail (START, FLOATING/ORDERED, scan elements, PARTITION, STOP) and understand how each flip-flop instance is listed.



Key Takeaway:

The header section sets the format, naming rules and top design name. It must be correct and consistent with your RTL, netlist and DEF to ensure successful scan chain processing in DFT and P&R tools.

Correct SCANDEF
→ Smooth DFT



SCANDEF File – Complete Example (Part 2/3)

SCANCHAINS Section – Chain Definition, Scan Elements, PARTITION and STOP

Describes with STA PARTITION

mimo_top.scandef (Part 2/3 – Scan Chain Definitions) Lines 26 – 70

```

26 #=====
27 # Scan chain definitions
28 #=====
29
30 SCANCHAINS 2 ; # Total 2 scan chains
31
32 #-----
33 # Chain1 : FFT block (RF domain)
34 #-----
35 - chain1
36 + START PIN scan_in1 ;
37 + FLOATING ;
38 fft_core/butterfly_reg_0 ( IN SI ) ( OUT Q ) ;
39 fft_core/butterfly_reg_1 ( IN SI ) ( OUT Q ) ;
40 fft_core/butterfly_reg_2 ( IN SI ) ( OUT Q ) ;
41 fft_core/butterfly_reg_3 ( IN SI ) ( OUT Q ) ;
42 fft_core/twiddle_reg_0 ( IN SI ) ( OUT Q ) ;
43 fft_core/twiddle_reg_1 ( IN SI ) ( OUT Q ) ;
44 fft_core/add_reg_0 ( IN SI ) ( OUT Q ) ;
45 fft_core/add_reg_1 ( IN SI ) ( OUT Q ) ;
46 + PARTITION clk_rf ;
47 + STOP PIN scan_out1 ;
48
49 #-----
50 # Chain2 : Channel estimation block (DSP domain)
51 #-----
52 - chain2
53 + START PIN scan_in2 ;
54 + FLOATING ;
55 channel_est/coeff_reg_0 ( IN SI ) ( OUT Q ) ;

```



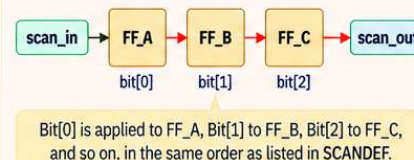
This is Part 2 of 3. It shows two scan chains with their scan elements. The SCANCHAINS section lists each chain with START, element list, PARTITION and STOP.

1 Syntax Breakdown (Chain Definition)

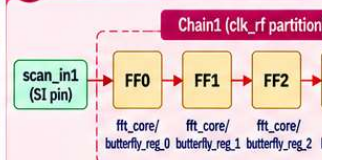
Keyword / Syntax	Meaning
SCANCHAINS <num> ;	Declares total number of scan chains.
- chain1	Start of a chain. Chain name (unique identifier).
+ START PIN <pin> ;	Entry point of the chain (must be a valid port).
+ FLOATING ;	Scan elements in this chain can be reordered by the tool (placement-aware).
<instance_name> (IN SI) (OUT Q) ;	Scan flip-flop instance and its scan pins.
+ PARTITION <name> ;	Chain belongs to this partition (e.g., clk_rf).
+ STOP PIN <pin> ;	Exit point of the chain (must be a valid port).

4 Why Instance Order Matters ?

- The order of instances defines the bit position in the scan chain.
- ATPG uses this order to create test patterns.
- If the order is wrong, good silicon may fail test (false failures).



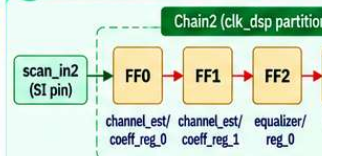
2 Example: Chain1 (FFT Block)



Key Points:

- scan_in1 is the START pin of chain1.
- All listed flip-flop instances are connected.
- Chain1 belongs to partition 'clk_rf'.
- scan_out1 is the STOP pin of chain1.

3 Example: Chain2 (Channel Estimation)



Key Points:

- scan_in2 is the START pin of chain2.
- Instances from different modules can be connected.
- Chain2 belongs to partition 'clk_dsp'.
- scan_out2 is the STOP pin of chain2.

5 Multiple Modules in a Chain

- A chain can contain flip-flops from different hierarchies.
- Example: fft_core, channel_est, equalizer can all be in the same chain.
- Instance names must match exactly with the netlist.
- This allows the tool to reorder across block boundaries.



Key Takeaway:

The SCANCHAINS section is the heart of the SCANDEF file. It lists each chain, the ordered scan elements, their scan pins, and the partition they belong to. The information must match your actual design.

Correct chain definition → Better results
→ Easier routing → Faster



SCANDEF File – Complete Example (Part 2/3)

Describe with STA

SCANCHAINS Section – Chain Definition, Scan Elements, PARTITION and STOP

1 Syntax Breakdown (Chain Definition)

Keyword / Syntax	Meaning
SCANCHAINS <num> ;	Declares total number of scan chains.
- chain1	Start of a chain. Chain name (unique identifier).
+ START PIN <pin> ;	Entry point of the chain (must be a valid port).
+ FLOATING ;	Scan elements in this chain can be reordered by the tool (placement-aware).
<instance_name> (IN SI) (OUT Q) ;	Scan flip-flop instance and its scan pins.
+ PARTITION <name> ;	Chain belongs to this partition (e.g., clk_rf).
+ STOP PIN <pin> ;	Exit point of the chain (must be a valid port).

2 Example: Chain1 (FFT Block)

Key Points:

- scan_in1 is the START pin of chain1.
- All listed flip-flop instances are connected to the chain.
- Chain1 belongs to partition 'clk_rf'.
- scan_out1 is the STOP pin of chain1.

3 Example: Chain2 (Channel Estimation Block)

Key Points:

- scan_in2 is the START pin of chain2.
- Instances from different modules can be in the same chain.
- Chain2 belongs to partition 'clk_dsp'.
- scan_out2 is the STOP pin of chain2.

4 Why Instance Order Matters?

- The order of instances defines the bit position in the scan chain.
- ATPG uses this order to create test patterns.
- If the order is wrong, good silicon may fail test (false failures).

5 Multiple Modules in a Chain

- A chain can contain flip-flops from different hierarchies.
- Example: fft_core, channel_est, equalizer can all be in the same chain.
- Instance names must match exactly with the netlist.
- This allows the tool to reorder across block boundaries.

This is Part 2 of 3. It shows two scan chains with their scan elements. The SCANCHAINS section lists each chain with START, element list, PARTITION and STOP.

SCANDEF File – Complete Example (Part 3/3)

PARTITION, MAXBITS, Additional Options and End of File

1 More SCANDEF Constructs

Construct	Example	Purpose / Meaning
ORDERED	+ ORDERED ;	- Preserve the relative order of listed scan elements. - Tool cannot reorder these elements.
PARTITION	+ PARTITION clk_ctrl ;	- Group of chains that may exchange flip-flops during reordering. - Typically chains in same clock domain or test constraints.
MAXBITS	+ MAXBITS 256 ;	- Specify maximum number of scan elements (bits) allowed in a chain (or segment). - Tool may split chain if limit is exceeded.
STOP	+ STOP PIN <pin> ;	End point of the chain.
END SCANCHAINS	END SCANCHAINS	- Marks the end of all chain definitions in the file. - No more chains after this statement.

2 ORDERED Example

The elements must remain in the same order.

Tool will not change the order. Useful for FSMs, counters or logic. Ensures correct test functionality.

3 MAXBITS Example

Limit the number of scan elements in a chain.

Tool keeps the chain size within limits. If more elements are present, it splits into multiple chains. Helps control test time, routing.

4 Partitioning Across Chains

Chains in the same partition (e.g., clk_dsp) can exchange flip-flops during scan reordering.

Typically chains with same clock domain. Improves placement and routing by giving the tool flexibility.

5 End of File

END SCANCHAINS

- Marks the end of the file.
- No additional options.
- Other options may appear after.

6 Complete SCANDEF Flow (Recap)

Key Takeaway:

The SCANCHAINS section is the heart of the SCANDEF file. It lists each chain, the ordered scan elements, their scan pins, and the partition they belong to. The information must match your actual design.

Correct chain definition → Easier routing → Faster

This is Part 3 of 3. It shows additional chains with ORDERED and MAXBITS examples.

and the end of the SCANCHAINS section.

**Key Takeaway:**

A complete SCANDEF file includes the header, all scan chains with their constructs (FLOATING/ORDERED, PARTITION, MAXBITS), and ends with **END SCANCHAINS**. It provides the scan chain structure to the physical implementation tool for placement-aware reordering.

Well defined scan chains →
Lower congestion → Faster t